© Victor Tan 2025

Angular Workshop
Lab 1: Data Binding
Exercises

1 LAB SETUP 1
2 GENERATING A NEW ANGULAR PROJECT 1
3 GENERAL STRUCTURE OF AN ANGULAR APP 2
4 TEMPLATE EXPRESSIONS AND INTERPOLATIONS 2
5 ANGULAR APP SCRIPTS AND THE DOM 3
6 PROPERTY BINDING 3
7 CLASS BINDING 3
8 STYLE BINDING 4
9 EVENT BINDING 4
9.1 TEMPLATE REFERENCE VARIABLES FOR EVENT BINDING 4
9.2 COMBINING INTERPOLATION, PROPERTY, CLASS AND EVENT BINDING 4
10 TWO WAY BINDING WITH NGMODEL 4

1 Lab setup

2 Generating a new Angular project

We will generate a separate Angular project to work with our exercises. We can then copy and paste code from the corresponding Angular project for the lab demo

In a separate command prompt (or shell terminal) or in an embedded terminal in Visual Studio Code inside the main folder on your machine for Angular projects, create a new Angular project with:

```
ng new exercisebinding
```

Press enter to accept the default values for all the question prompts that follow regarding stylesheet format (CSS) and enabling Server side rendering (No)

```
G:\temp\workshoplabs>ng new bindingdemo
? Which stylesheet format would you like to use? CSS            [
https://developer.mozilla.org/docs/Web/CSS                    ]
? Do you want to enable Server-Side Rendering (SSR) and Static Site
Generation (SSG/Prerendering)? (y/N)
```

Navigate into the root project folder from the terminal with:

```
cd exercisebinding
```

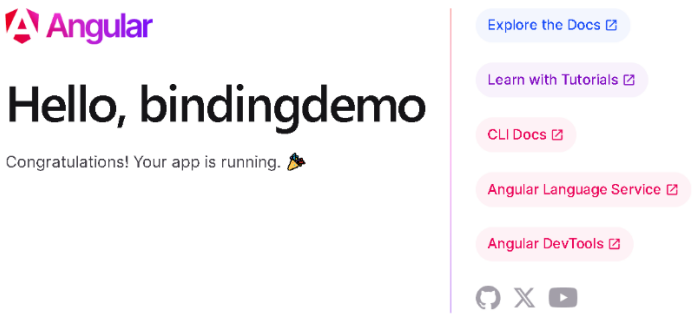
Build the app and serve it via Angular's live development server by typing:

```
ng serve
```

If the live development server for the other Angular project is still running, Angular will prompt you on using a different port for this instance:

```
Port 4200 is already in use.
Would you like to use a different port? Yes
```

Select Yes, and the live development server will pick a random free port on your machine to serve your app on. You can then open a browser tab at this port and you should be able to see the default landing page for all new autogenerated Angular projects.



3 General structure of an Angular app

4 Template expressions and interpolations

Q1: Add in a property to the root component called `myAge` and give it any random value

Q2: Display the value of this property with a random string (for e.g. `I am 60 years old`) in the template HTML

Q3: Add in a method called `getAgeMessage` which returns a string depending on the value of `myAge`. The content of the string returned is:

`myAge` less than 30: I am sooo young
`myAge` between 31 and 50 : I am middle aged
`myAge` more than 51: I am soooo old.

Q4: Using this method, display a message similar to the following in the template:
`I am 40 years old, and I am middle aged`
Change the value of `myAge` to check that the correct messages are displayed for the correct age.

5 Angular app scripts and the DOM

6 Property binding

Q1: Add in two properties `socialMediaSite` (which holds any random valid URL) and `myOpenStyle` (which holds the value of either `_self` or `_blank` (see Q6 from Topic 4) to the root component

Q2: Add in an <a> element into the template which uses these two properties in the syntax of property binding. Check that it works properly.

7 Class binding

First Part: (after `app.component-v3.ts, app.component-v3.html`)

Q1: Create a element and nest a <p> with some text within this . The CSS Box model provides a set of properties to delineate an element and its child elements. You can use a variety of properties to control the border look for the . Set a suitable `padding` for the .

Q2: Add 2 different classes in the template CSS which sets different values for the `border-style` property, another 2 different classes which sets different values for the `border-color` property, and another 2 different classes which sets different values for the `border-width` property. Give these classes any suitable names.

Q3: Add any one of these 6 classes to the , and then use single binding to add another different class to the , depending on a Boolean property in the component.

Q4: Add any 2 or more of these 6 classes to the , and then use single binding to remove one of these classes, depending on a Boolean property in the component.

Second Part:

Reuse the same and classes from First Part: Q1 and Q2

Q5: Add any one of the 6 classes to the , and use a string to add another 2 more of these classes to the

Q6: Add any one of the 6 classes to the , and use an object to add another 2 more of these classes to the while removing the existing class.

Q7. Add any one of the 6 classes to the , and use an array of strings to add another 2 more of these classes to the

8 Style binding

Q1: Create a <p> element and use single style binding to set its `font-family` property.

Q2: Create a element and nest a <p> with some text within this . Use a string containing the relevant properties to style the

Q3: Create another element and nest a <p> with some text within this . Use an object containing the relevant properties to style this

9 Event binding

Q1: Add in 3 radio buttons, each with different values.

Q2. Provide event binding for the click event for all these 3 radio buttons to the same component method

Q3. Implement this component method so that it logs the value of the particular radio button clicked.

9.1 Template reference variables for event binding

Repeat Q1 - Q3 from the previous section, but implement the event binding using template reference variables instead.

9.2 Combining interpolation, property, class and event binding

Q1. Using any 2 or more HTML elements of your choice (text field, normal button, radio button, check box, paragraph, etc) create a scenario of your design that combines interpolation, property, class and event binding.

10 Two way binding with ngModel

Q1. Create a radio button. Use banana-in-the-box syntax to perform 2 way binding between the button value and a component property and display the property via interpolation.

© Victor Tan 2025